

Module 8: Security and Reliability

Guardrails for AI-assisted systems

Module 8: Security and Reliability

Primary sources: Osmani, Chapter 8; Thomas & Hunt, Estimating, Refactoring, and Stay Safe Out There.

Learning Outcomes

- Identify security risks in AI-generated code.
 - Analyze reliability challenges.
 - Evaluate safety, performance, and complexity tradeoffs.
 - Propose strategies to improve robustness.
-

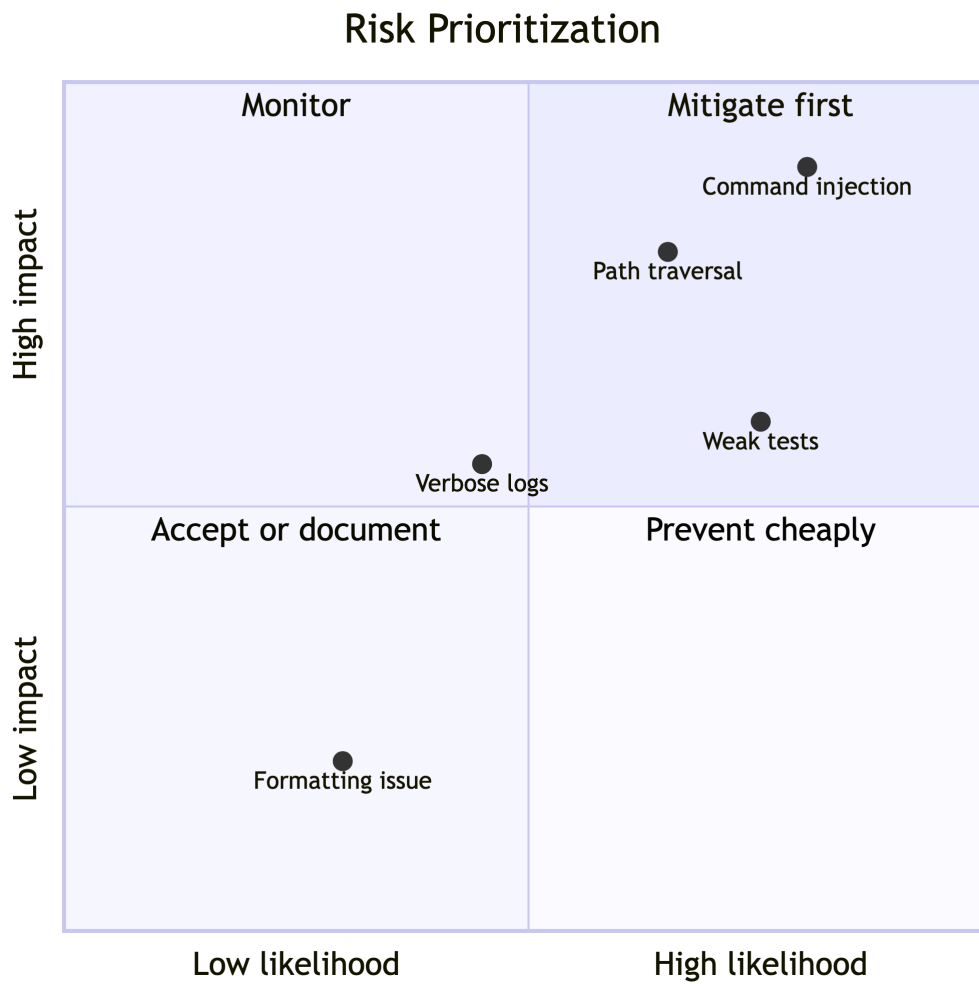
AI-Specific Risk Pattern

AI often generates code optimized for the visible task, not the threat model.

Risks include:

- Missing authorization checks.
 - Insecure defaults.
 - Unsafe command execution.
 - Overbroad file access.
 - Dependency confusion.
 - Logging secrets.
-

Risk Matrix



Security Review Areas

- Input validation.
- Authentication and authorization.
- Secrets handling.
- Dependency versions.
- Path traversal.
- Command injection.
- Network access.

- Audit logs.
-

Reliability Review Areas

- Timeouts.
 - Retries and backoff.
 - Deterministic test paths.
 - Clear failure states.
 - Idempotent operations.
 - Recoverable artifacts.
 - Observability.
-

Refactoring AI Code

Refactoring turns generated code into maintainable code without changing behavior.

Use tests first, then improve:

- Names.
 - Duplication.
 - Boundaries.
 - Error handling.
 - Configuration.
-

Estimating Risk

Risk ratings should be concrete.

Use a simple scale:

- Likelihood: low, medium, high.
 - Impact: low, medium, high.
 - Mitigation: specific action.
 - Evidence: test, review, log, or policy.
-

Safeguard Examples

- Reject paths outside workspace.
 - Require confirmation before writes.
 - Disable shell commands by default.
 - Redact environment variables from logs.
 - Add tests for malicious inputs.
 - Fail closed when model output is malformed.
-

Lab 4 Final Connection

Your submission should include:

- Specific failure modes.
 - Risk ratings.
 - Concrete mitigations.
 - At least two safeguards or safety tests.
 - Production readiness plan.
 - Copilot-assisted analysis corrections.
-

Practice Activity

Threat-model your Java agent:

- What input could cause harm?
 - What tool could be misused?
 - What output could mislead a user?
 - What safeguard would fail safely?
-

Takeaways

- Generated code is not security-reviewed code.
- Reliability requires explicit design for failure.
- Safeguards must be implemented and tested, not merely described.